

CS 219 Operating Systems

Spring 2025-26

Quiz 2 (20 marks)

A

Name: _____ Roll No.: _____

Q1. State TRUE or FALSE and justify. [7 x 1 = 7 marks] **No correct justification, no marks.**

- a. If a process is executing an infinite loop, e.g., via a **while(1) {}** sequence, the OS scheduler will never be invoked.

FALSE.

On an interrupt (IO device or timer), control switches to kernel mode. Once in kernel mode the scheduler can be invoked, e.g., by the timer interrupt handler to decide if a new process has to be scheduled on to the CPU.

- b. The **kernel context** of a process is only needed for a newly created/forked process, else user context saved during switch from user to kernel mode is sufficient for scheduling across processes by the scheduler.

FALSE.

Once a process switches to kernel mode, execution in the kernel mode in the context of/for this process builds/updates kernel context — CPU registers, state of kernel stack etc. For a process to be de-scheduled, this kernel execution context has to be stored, so that when the process is rescheduled, and it restarts where it left off in the kernel mode, the kernel context can be used to restore context of execution.

- c. The weighted round robin scheduler — assign timeslice on the CPU for a process proportional to its weight, is an example of a work conserving scheduler.

TRUE.

WRR is a work conserving scheduler since it will always schedule READY processes if the CPU is available, and never leaves the CPU idle. WRR only differentiates the extent of CPU available to scheduled processes.

- d. User space threads executing concurrently, share the same page table for all user virtual addresses and hence will eventually **always** end up in a race condition.

FALSE.

Different threads can access different regions of shared memory/address space and not cause any race conditions.

- e. The following instruction, which compares the value of register **eax** with a constant is atomic — **lock; cmp eax, 1**

TRUE.

Each CPU has its own registers, hence it will be atomic since registers are cpu private. Even if the instruction is treated as an indirect addressing (address is in register), answer is true as the **lock;** prefix locks the address bus for use till the instruction is completed.

- f. A semaphore can be used as a mutex lock.

TRUE.

A binary semaphore as a mutex lock.

- g. With multiple CPUs, schedulers executing concurrently is only possible if each CPU has its own separate runqueue (queue of ready processes).

FALSE.

A single runqueue can also be used for concurrently scheduling processes on multiple CPUs as long as locks are used to ensure that two CPUs do not dequeue and schedule the same process.

Q2 a. Describe **two** examples of kernel work that is instantiated as schedulable entities — kernel threads/daemons (kernel “processes”) which are scheduled by the scheduler. In particular describe the nature of work, why such work are good examples to be instantiated as kernel threads/daemons and the good events/situations to schedule them. **[4 marks]**

Any **two** answers from the following list or others with correct reasons. |

Names of the daemons are not required as part of the answer.

- **kswapd** is a kernel daemon used for managing virtual memory by reclaiming pages by swapping them to swap when free memory drops below a **low** threshold.

- **kmigrationd** — a kernel daemon invoked periodically or on imbalanced per-CPU runqueues. The kernel thread (could be per-cpu kernel thread) checks other runqueues to push or pull entities between runqueues.

- a kernel thread that periodically wakes up to perform lazy cleanup of TERMINATED processes, e.g., reclaim physical memory via page table cleanup etc.

- **ksoftirqd** are a set of per-CPU kernel threads that handle deferred interrupts when the system is under low load, such as intense network traffic or Disk I/O.

- **kworker** processes are kernel threads that handle background tasks, specifically managing workqueues for hardware interrupts, timers, and I/O operations.

Q2.b. What are **two** (different types of) system calls, when invoked by a user process, will **always** call the scheduler as part of their implementation? Explain why. **[2 marks]**

Any TWO.

(i) exit

process is done, cannot be rescheduled and hence scheduler has to be invoked.

(ii) sleep/pause

process indicates block/wait till condition is satisfied. cannot be rescheduled immediately and hence the scheduler has to be invoked.

(iii) yield

process indicates intent to give up CPU, cannot be rescheduled immediately and hence the scheduler has to be invoked.

(iv) any blocking system call — read, write, mutex lock, which has to wait for a condition.

Q2.c. Spinlocks when implemented in the kernel for kernel threads require to disable interrupts. Explain why, with an example.

Spinlocks when implemented in the user space for user space threads on a user space lock variable **do not** require to disable interrupts. Explain why, with an example. **[3 marks]**

In the kernel, spinlocks must disable interrupts to prevent a deadlock situation with nested interrupts. For example, when a CPU is executing `yield()` — to switch processes, the kernel acquires the process spinlock (`p->lock`). If a timer interrupt occurred at that same moment and the interrupt handler also tried to access the same process or call the scheduler, it would spin forever waiting for `p->lock` and not return.

In user space, spinlocks do not need to disable interrupts because the OS uses preemptive scheduling to prevent deadlocks.

For example, if User Thread A acquires a lock and is then descheduled by a timer interrupt, the kernel might switch to Thread B. If Thread B tries to acquire that same lock, it will simply spin and waste its time slice. However, because interrupts remain enabled, the timer interrupt will eventually fire again, allowing the kernel to preempt Thread B and put Thread A back on the CPU (eventually). Thread A then finishes its work and releases the lock, allowing the system to progress.

Q2.d. Write the pseudo-code for implementing the **barrier** synchronization primitive using condition variables. Following is the basic barrier object/structure and the functions to be

implemented. Declare variables as needed and not to use the pthread API. Assume that the process calling barrier can be identified using the function `myproc()` [4 marks]

<pre>typedef struct { // details needed! }barrier; barrier *b;</pre>	<pre>void init(barrier *b); void barrier(barrier *b); void sleep(barrier *b); void wakeup(barrier *b);</pre>
--	--

Row 1: 0.5 + 0.5

Row 2: 1 + 1

Row 3: 1

<pre>typedef struct { int value; spinlock L; }barrier;</pre>	<pre>void init(barrier* b, int val) { b->val=val; }</pre>
<pre>void barrier(barrier *b) { spinlock(L); b->val = b->val - 1; if(b->val > 0) sleep(b); wakeup(b); spinunlock(L); }</pre>	<pre>void sleep(barrier *b,) { p=myproc(); p->state = WAITING; p->cond = b; // wait condition spinunlock(); scheduler(b->L); spinlock(b->L); }</pre>
<pre>void wakeup(barrier *b) { for all waiting processes p { if (p->cond==b) { p->state = READY; p->cond = 0; // reset wait condition } } }</pre>	